CODE-DOKUMENTATION

Willhaben Scraper & Price Predictor

Eine Flask-Webanwendung zum Scrapen von Inseraten auf willhaben.at
mit integrierter Preisprognose auf Basis von Machine Learning

Sprache: Python 3.14+
Framework: Flask 3.x mit Flask-SQLAlchemy
Datenbank: SQLite (konfigurierbar)
ML-Bibliothek: scikit-learn (Ridge Regression)
Architektur: MVC mit separater Service-Schicht
Tests: pytest mit pytest-flask

1. Projektübersicht

Willhaben Scraper ist eine Flask-Webanwendung, die Inserate von willhaben.at (dem größten österreichischen Kleinanzeigenportal) automatisch abruft, in einer Datenbank speichert und auf Basis der gesammelten Preisdaten eine Preisprognose mittels Machine Learning erstellt.

1.1 Hauptfunktionen

- Keyword-basierte Suche auf willhaben.at (mehrere Seiten)
- Automatisches Parsen von Titel, Preis, Standort, Beschreibung und Bild-URL
- Dual-Strategie: JSON-Extraktion aus Next.js-Payload, Fallback auf HTML-Parsing
- Persistente Speicherung aller Ergebnisse in SQLite-Datenbank
- Katalogansicht mit Filtern (Name, Preis, Ort, Sortierung) und Paginierung
- Preisprognose via polynomial Ridge Regression mit einstellbarer Perzentile

1.2 Technologie-Stack

Komponente	Technologie / Bibliothek
Web-Framework	Flask 3.x
Datenbank ORM	Flask-SQLAlchemy 3.x
HTML-Parser	BeautifulSoup4 + lxml
HTTP-Client	requests
Machine Learning	scikit-learn (Ridge, PolynomialFeatures, Pipeline)
Numerische Berechnungen	NumPy
Templating	Jinja2 (in Flask integriert)

Testing	pytest + pytest-flask
Paket-Manager	uv (mit pyproject.toml)

2. Projektstruktur

Das Projekt folgt dem MVC-Muster (Model-View-Controller) mit einer zusätzlichen Service-Schicht für komplexe Geschäftslogik:

```
scraper-main/
├── run.py                          # Einstiegspunkt der Anwendung
├── config.py                       # Konfigurationsklassen
├── requirements.txt               # Abhängigkeiten
├── pyproject.toml                 # Projektmetadaten (uv)
├── app/
│   ├── __init__.py              # App-Factory: create_app()
│   ├── models/
│   │   └── __init__.py          # MODEL: SearchQuery, Product
│   ├── controllers/
│   │   ├── main.py              # CONTROLLER: Startseite
│   │   └── scraper.py           # CONTROLLER: Scraping-Routen
│   ├── services/
│   │   ├── willhaben_scraper.py # SERVICE: Web-Scraping
│   │   └── price_predictor.py   # SERVICE: ML-Preisprognose
│   ├── templates/
│   │   ├── base.html            # VIEW: Basis-Template
│   │   ├── index.html           # VIEW: Startseite
│   │   ├── products.html        # VIEW: Produktliste
│   │   ├── prediction.html      # VIEW: Preisprognose
│   │   ├── catalog.html         # VIEW: Gesamtkatalog
│   │   └── catalog_prediction.html  # VIEW: Katalog-Prognose
│   └── static/css/main.css      # CSS-Stylesheet
└── tests/
    ├── conftest.py              # Test-Fixtures
    ├── test_models.py           # Datenbank-Tests
    ├── test_predictor.py        # ML-Tests
    ├── test_routes.py           # HTTP-Routen-Tests
    └── test_scraper.py          # Scraper-Tests
```

2.1 MVC-Architektur

Schicht	Dateien & Aufgabe
Model	app/models/__init__.py – Datenbankstruktur und -zugriff
View	app/templates/*.html – HTML-Darstellung für den Browser
Controller	app/controllers/*.py – HTTP-Anfragen entgegennehmen, Daten verarbeiten, an View übergeben
Service	app/services/*.py – Geschäftslogik (Scraping, ML) ausgelagert aus dem Controller

3. Konfiguration (config.py)

Die Konfiguration ist in drei Klassen aufgeteilt, die alle von einer gemeinsamen Basisklasse erben. Dies ist ein gängiges Flask-Muster für umgebungsabhängige Einstellungen.

3.1 Klasse: Config (Basisklasse)

Attribut	Beschreibung
SECRET_KEY	Geheimschlüssel für Flask-Sessions. Wird aus Umgebungsvariable gelesen, fällt auf dev-secret-key zurück.

SQLALCHEMY_TRACK_MODIFICATIONS	Deaktiviert (False) das kostspielige Tracking von Objektänderungen in SQLAlchemy.
SCRAPE_REQUEST_TIMEOUT	HTTP-Timeout in Sekunden für Willhaben-Anfragen. Standard: 10 Sekunden.
SCRAPE_MAX_PAGES	Maximale Anzahl an Seiten, die pro Suche gescrapt werden. Standard: 5.
WILLHABEN_SEARCH_URL	Basis-URL für die Willhaben-Marktplatz-Suche.

3.2 Unterklassen

- DevelopmentConfig: DEBUG=True, SQLite-Datei scraper_dev.db
- TestingConfig: TESTING=True, In-Memory-SQLite (:memory:), CSRF deaktiviert
- ProductionConfig: DEBUG=False, SQLite-Datei scraper.db

Die aktive Konfiguration wird beim Start über die Umgebungsvariable FLASK_ENV gewählt:

```
# Beispiel: Produktionsumgebung aktivieren
export FLASK_ENV=production
python run.py
```

4. Anwendungskern (app/__init__.py)

Diese Datei enthält die App-Factory-Funktion create_app() sowie die Datenbankinitialisierung. Das App-Factory-Pattern ermöglicht es, mehrere App-Instanzen mit unterschiedlichen Konfigurationen zu erstellen – was besonders für Tests nützlich ist.

4.1 Funktion: create_app(config_name)

Erstellt und konfiguriert die Flask-Anwendung. Ablauf:

- Konfiguration laden (aus config.py, per config_name ausgewählt)
- SQLAlchemy-Instanz initialisieren (db.init_app)
- Blueprints registrieren (main_bp und scraper_bp)
- Datenbanktabellen erstellen (db.create_all)
- Schema-Migrationen prüfen (_ensure_schema)

4.2 Funktion: _ensure_schema()

Eine leichtgewichtige Schema-Migrationslogik für bestehende Datenbanken. Sie prüft, ob die Spalte image_url in der Tabelle products vorhanden ist, und fügt sie per ALTER TABLE hinzu, falls sie fehlt. Dies ermöglicht Updates ohne Datenverlust.

> ⚠️ Wichtiger Hinweis: Diese manuelle Migrationslösung ist für einfache Fälle ausreichend. Für größere Projekte mit häufigen Schema-Änderungen empfiehlt sich Flask-Migrate (basierend auf Alembic).

5. Datenbankmodelle (app/models/__init__.py)

Die Models repräsentieren die Datenbankstruktur. Beide Klassen erben von db.Model (Flask-SQLAlchemy) und werden automatisch als Tabellen in der konfigurierten Datenbank angelegt.

5.1 Klasse: SearchQuery

Repräsentiert einen einzelnen Suchvorgang. Tabelle: search_queries

Spalte	Typ & Beschreibung
id	Integer, Primärschlüssel, auto-increment
keyword	String(255), nicht null, indiziert – der eingegebene Suchbegriff
created_at	DateTime, nicht null – Zeitpunkt der Suche (UTC, automatisch gesetzt)
products	Beziehung (1:n) zu Product, cascade delete-orphan

5.2 Klasse: Product

Repräsentiert ein einzelnes Inserat von willhaben.at. Tabelle: products

Spalte	Typ & Beschreibung
id	Integer, Primärschlüssel
search_query_id	Integer, Fremdschlüssel auf search_queries.id, indiziert
title	String(500), nicht null – Titel des Inserats
price	Float, nullable – Preis in Euro (None wenn nicht angegeben)
location	String(255), nullable – Standortangabe des Verkäufers
url	String(1000), nullable – Direkt-Link zum Inserat
image_url	String(1000), nullable – URL des Vorschaubilds
description	Text, nullable – Kurzbeschreibung
scraped_at	DateTime, nicht null – Zeitpunkt des Scrapings (UTC)

5.3 Methode: Product.to_dict()

Konvertiert ein Product-Objekt in ein Python-Dictionary. Nützlich für JSON-Serialisierung und Tests. Das scraped_at-Feld wird als ISO-8601-String ausgegeben.

6. Controller

6.1 Main Controller (app/controllers/main.py)

Enthält nur eine Route und ist als Blueprint main_bp registriert.

Route: GET /

Die Startseite der Anwendung. Lädt die 10 neuesten SearchQuery-Einträge aus der Datenbank und übergibt sie an das Template index.html. Zeigt auch den konfigurierten Standard-Wert für max_pages im Suchformular.

6.2 Scraper Controller (app/controllers/scraper.py)

Das Herzstück der Anwendung. Registriert als Blueprint scraper_bp mit URL-Präfix /scraper. Enthält alle Scraping-, Produkt- und Prognoserouten.

Hilfsfunktionen

- _get_search_or_404(search_id): Lädt eine SearchQuery per ID, gibt HTTP 404 zurück wenn nicht gefunden.

- _get_percentile_arg(): Liest den percentile-Query-Parameter aus der URL, klemmt ihn auf 1.0–99.0.
- _get_catalog_filters(): Liest alle Filterparameter der Katalogansicht (search, min_price, max_price, location, sort).
- _apply_catalog_filters(query, filters): Wendet die Filter als SQLAlchemy-Filterketten auf die Datenbankanfrage an.
- _apply_catalog_sorting(query, sort_by): Sortiert die Anfrage nach newest, oldest, price_asc oder price_desc.

Route: POST /scraper/search – Neue Suche

Diese Route führt den vollständigen Scraping-Vorgang durch:

- Keyword und max_pages aus dem Formular lesen
- scrape_willhaben() aus dem Service aufrufen
- SearchQuery-Objekt anlegen und in DB speichern (flush für ID)
- Für jedes Ergebnis ein Product-Objekt erstellen und speichern
- Commit und Weiterleitung auf die Produktliste

Route: GET /scraper/products/<search_id> – Produktliste

Lädt alle Produkte einer SearchQuery aus der Datenbank, sortiert nach Preis aufsteigend (NULL-Werte zuletzt), und übergibt sie an products.html.

Route: GET /scraper/predict/<search_id> – Preisprognose

Lädt alle Produkte einer Suche und ruft predict_price() auf. Der gewünschte Perzentilwert kann per Query-Parameter percentile angepasst werden (Standard: 50).

Route: GET /scraper/catalog – Gesamtkatalog

Zeigt alle Produkte aus der Datenbank mit Filter- und Sortierfunktion. Unterstützt Paginierung (20 Einträge pro Seite). Gibt auch eine Liste aller Suchen für die Statistik-Box mit.

Route: GET /scraper/catalog/predict – Katalog-Prognose

Preisprognose basierend auf den aktuell aktiven Katalogfiltern. Lädt alle gefilterten Produkte (ohne Paginierung) und ruft predict_price() auf.

Route: POST /scraper/delete/<search_id> – Suche löschen

Löscht eine SearchQuery und (dank cascade="all, delete-orphan") automatisch alle zugehörigen Produkte. Leitet danach auf die Startseite weiter.

□

7. Scraper Service (app/services/willhaben_scraper.py)

Der technisch komplexeste Teil des Projekts. Implementiert das Abrufen und Parsen der Willhaben-Suchergebnisse in zwei Strategien.

7.1 Konstanten

- HEADERS: Browser-User-Agent und Accept-Header für realistische HTTP-Anfragen (verhindert einfache Bot-Erkennung)

- WILLHABEN_BASE: Basis-URL https://www.willhaben.at
- WILLHABEN_SEARCH: Vollständige Such-URL zum Marktplatz

7.2 Hilfsfunktionen

_normalize_url(url)

Konvertiert relative URLs in absolute. Behandelt drei Fälle:

- URL beginnt mit // → https: wird vorangestellt
- URL beginnt mit / → WILLHABEN_BASE wird vorangestellt
- Absolute URL → unvereändert zurückgegeben

_parse_price(text)

Extrahiert einen Float-Preis aus europäischen Preisstrings wie "€ 1.234,56". Korrekte Reihenfolge:

- Schritt 1: Alle Nicht-Ziffer/Nicht-Komma/Nicht-Punkt-Zeichen entfernen (Regex)
- Schritt 2: Tausenderpunkte entfernen (replace(".", ""))
- Schritt 3: Dezimalkomma in Punkt umwandeln (replace(",", "."))
- Schritt 4: float() Konvertierung

> 🐛 Bug-Hinweis: In der Originalversion war die Reihenfolge falsch (replace vor Regex), was zu fehlerhaften Preisen führte. Die korrigierte Version wendet zuerst die Regex an.

_extract_image_url(article)

Sucht im article-Tag nach dem besten Bild-URL. Prüft der Reihe nach die Attribute: data-src, src, data-original, data-image. Falls keines gefunden, wird srcset oder data-srcset als Fallback verwendet (erster Kandidat).

_parse_listing(article)

Parst ein einzelnes <article>-Tag einer Suchergebnisseite und gibt ein Dictionary zurück. Verwendet data-testid-Attribute als primäre Selektoren (z.B. ad-detail-link, ad-price, ad-location), mit CSS-Klassen als Fallback.

7.3 Extraktionsstrategie 1: __NEXT_DATA__ JSON

_extract_next_data_products(html)

Willhaben nutzt Next.js. Das bedeutet, die vollständigen Seitendaten liegen als JSON in einem <script id="__NEXT_DATA__">-Tag eingebettet. Diese Funktion:

- Findet das Script-Tag per Regex
- Parst den JSON-Payload
- Navigiert zu props.pageProps.searchResult.advertSummaryList.advertSummary
- Extrahiert für jedes Inserat: Titel (HEADING), Preis (PRICE), Standort (LOCATION), Beschreibung (BODY_DYN), Link (adDetailLink) und Bild-URL

> 🐛 Bug-Hinweis: In der Originalversion fiel die Beschreibung (BODY_DYN) auf

advert.get("description") zurück – das gleiche Feld wie der Titel. Die Korrektur verwendet einen leeren String als Fallback.

_extract_advert_image_url(advert)

Extrahiert die Bild-URL aus einem einzelnen Inserat-Dictionary des JSON-Payloads. Prüft in dieser Priorität: mainImageUrl, thumbnailImageUrl, referenceImageUrl.

_merge_image_urls(primary_results, html_results)

Kombiniert die Ergebnisse beider Extraktionsmethoden: Falls ein JSON-Ergebnis keine Bild-URL hat, wird die entsprechende URL aus den HTML-Ergebnissen nachgetragen (Lookup zuerst per URL, dann per Titel).

7.4 Extraktionsstrategie 2: HTML-Fallback

_extract_html_products(html)

Parst die HTML-Seite direkt mit BeautifulSoup (lxml-Parser). Sucht zunächst alle <article>-Tags, fällt auf CSS-Selektoren [data-testid="ad-card"] und li.search-result zurück. Gibt eine gefilterte Liste zurück (nur Einträge mit Titel).

7.5 Hauptfunktion: scrape_willhaben(keyword, max_pages, timeout)

Orchestriert den gesamten Scraping-Prozess:

- Iteriert über die Seiten 1 bis max_pages
- Baut die paginierten URLs mit requests.Request().prepare().url auf
- Versucht zuerst die __NEXT_DATA__-Extraktion
- Falls Bilder fehlen: _merge_image_urls() aufrufen
- Falls JSON leer: HTML-Fallback versuchen
- Falls auch HTML leer: Schleife beenden (keine weiteren Seiten)
- Pause von 1 Sekunde zwischen den Seiten (rate limiting)
- Logging aller relevanten Schritte

□

8. Price Predictor Service (app/services/price_predictor.py)

Implementiert die Machine-Learning-basierte Preisprognose auf Basis der gesammelten Inserat-Preise.

8.1 Funktion: _prices_array(products)

Hilfsfunktion, die aus einer Liste von Product-Objekten oder Dictionaries ein sortiertes NumPy-Array gültiger Preise extrahiert. Unterstützt sowohl Objekte (product.price) als auch Dictionaries (product["price"]). Filtert None-Werte und Preise ≤ 0 heraus.

8.2 Hauptfunktion: predict_price(products, percentile)

Berechnet eine Preisprognose auf Basis der vorhandenen Inserat-Preise. Eingabe: Liste von Product-Objekten oder Dictionaries, gewünschte Perzentile (Standard: 50.0).

Algorithmus (Schritt für Schritt)

- Schritt 1 – Daten vorbereiten: _prices_array() aufrufen, sortiertes Float-Array erstellen
- Schritt 2 – Mindestprüfung: Weniger als 3 gültige Preise → None-Werte zurückgeben
- Schritt 3 – Ausreißer entfernen: Werte unter dem 5. und über dem 95. Perzentil werden gefiltert
- Schritt 4 – Statistiken berechnen: Mean, Median, Min, Max auf den bereinigten Daten
- Schritt 5 – Modell trainieren: X = linspace(0, 1), Y = sortierte Preise. Polynomial Regression (Grad 2) + Ridge Regression (alpha=1.0) als sklearn Pipeline
- Schritt 6 – Vorhersage: Eingabe ist percentile/100.0 (z.B. 0.5 für 50. Perzentile)
- Schritt 7 – Rückgabe: Dictionary mit predicted_price, mean, median, min, max, count, percentile

Rückgabeformat

Schlüssel	Typ & Bedeutung
predicted_price	float – Vorhergesagter Preis an der gewünschten Perzentile
mean	float – Durchschnittspreis (bereinigt)
median	float – Medianpreis (bereinigt)
min	float – Minimalpreis (bereinigt)
max	float – Maximalpreis (bereinigt)
count	int – Anzahl der verwendeten Preise
percentile	float – Die gewählte Perzentile (Eingabewert)

💡 Warum polynomial Ridge Regression? Der Ansatz modelliert die Preisverteilung als Kurve statt als gerade Linie. Ridge Regression (L2-Regularisierung) verhindert Überanpassung bei kleinen Datensätzen. Das Ergebnis ist robuster als ein einfacher Percentile-Lookup.

9. Templates (app/templates/)

Alle Templates erben von base.html via Jinja2-Vererbung ({% extends "base.html" %}).

9.1 base.html – Basis-Layout

Definiert das Grundgerüst aller Seiten:

- HTML5-Grundstruktur mit Sprachattribut lang="de"
- Navbar mit Link zur Startseite und zum Katalog
- Flash-Message-Bereich (Erfolgs-, Warnungs- und Fehlermeldungen)
- {% block content %} Platzhalter für Seiteninhalt
- Footer mit Projekttitel
- CSS eingebunden via url_for("static", ...)

9.2 index.html – Startseite

Zeigt das Suchformular mit Keyword-Eingabe und max_pages-Feld. Darunter eine Tabelle der letzten 10 Suchen mit Links zu Produktliste, Prognose und Löschfunktion.

9.3 products.html – Produktliste

Zeigt alle Produkte einer Suche als Card-Grid. Jede Karte enthält: Produktbild (lazy loading), verlinkten Titel, Preis (formatiert mit 2 Dezimalstellen), Standort, Kurzbeschreibung (max. 150 Zeichen) und Scraping-Zeitpunkt.

9.4 prediction.html – Preisprognose

Zeigt das Prognoseergebnis für eine spezifische Suche. Enthält ein Formular zur Anpassung der Perzentile. Wenn genügend Daten vorhanden: Hervorgehobener Preis und Statistik-Tabelle. Sonst: Warnmeldung mit aktueller Datenlage.

9.5 catalog.html – Gesamtkatalog

Die umfangreichste View. Enthält Statistik-Box (Gesamt-Produkte, Anzahl Suchen), Filter-Formular mit allen Parametern, paginierten Produkt-Grid und Paginierungsnavigation. Der "Preisprognose"-Button verwendet formaction, um das Filterformular an catalog/predict zu senden.

9.6 catalog_prediction.html – Katalog-Prognose

Zeigt die Preisprognose für die aktuellen Katalogfilter. Listet die aktiven Filter als Übersicht auf. Gleiche Ergebnisdarstellung wie prediction.html, aber mit Katalog-Kontext.

□

10. Tests (tests/)

10.1 Test-Infrastruktur (conftest.py)

Definiert drei pytest-Fixtures:

- app (session-scoped): Erstellt eine Flask-Instanz mit TestingConfig (In-Memory-SQLite). Alle Tabellen werden am Anfang erstellt und am Ende gelöscht.
- db (function-scoped): Gibt die Datenbankinstanz zurück. Nach jedem Test: Session bereinigen, alle Tabellen löschen und neu erstellen (sauberer Zustand).
- client (function-scoped): Gibt den Flask Test-Client zurück.

10.2 test_models.py – Datenbank-Tests

Testfall	Prüft
test_create_search_query	SearchQuery wird korrekt angelegt (ID, Keyword, Datum)
test_create_product	Product mit allen Feldern wird korrekt gespeichert
test_product_to_dict	to_dict() gibt alle Felder korrekt zurück
test_cascade_delete	Löschen einer SearchQuery löscht auch alle Products

10.3 test_predictor.py – ML-Tests

Testfall	Prüft
test_predict_returns_none_for_empty	Leere Liste → None-Werte
test_predict_returns_none_for	Weniger als 3 Preise → None-Werte

_too_few	
test_predict_median_price	Prognosewert liegt zwischen Min und Max
test_predict_statistics	Mean und Median korrekt für bekannte Daten
test_predict_ignores_none_prices	None-Preise werden ignoriert
test_predict_low_percentile_lower_than_high	Niedrige Perzentile < hohe Perzentile

10.4 test_scraper.py – Scraper-Tests

Testfall	Prüft
test_parse_price_euro_format	Europäisches Format "€ 1.234,56" wird korrekt geparst
test_parse_price_comma_format	"150,00 €" wird korrekt geparst
test_parse_price_none_on_empty	Leerer String und None liefern None
test_parse_price_none_on_vhb	"VHB" (Verhandlungsbasis) liefert None
test_parse_listing_basic	HTML-Artikel wird korrekt in Dictionary geparst
test_extract_next_data_products_includes_image_url	__NEXT_DATA__ JSON wird vollständig geparst
test_scrape_willhaben_http_error	Netzwerkfehler führt zu leerer Liste
test_scrape_willhaben_empty_page	Seite ohne Inserate beendet Scraping
test_scrape_willhaben_parses_articles	Gemockte Seite wird korrekt geparst

10.5 test_routes.py – HTTP-Routen-Tests

Testet alle HTTP-Endpunkte mit dem Flask Test-Client und gemockten Services (unittest.mock.patch). Wesentliche Testfälle:

- Startseite gibt HTTP 200 zurück und enthält "Willhaben"
- POST /scraper/search mit leerem Keyword gibt Redirect (302)
- POST /scraper/search mit Keyword scrapt, speichert und redirectet
- max_pages-Parameter wird korrekt an scrape_willhaben() übergeben
- Produktliste zeigt Produkte und Bilder
- Prognoseseiten geben 200 zurück (genügend und zu wenig Daten)
- Katalog enthält Link zu /scraper/catalog/predict
- Löschen einer Suche gibt 302 zurück und entfernt Eintrag aus DB
- Nicht vorhandene search_id gibt 404 zurück

11. Vollständiger Datenfluss

Das folgende Diagramm zeigt den kompletten Weg einer Suchanfrage durch das System:

```
Nutzer gibt "iPhone 14" ein und klickt "Jetzt scrapen"

1. Browser → POST /scraper/search (keyword="iPhone 14", max_pages=5)

2. Controller (scraper.py)
```

```
  ├─ Liest keyword + max_pages aus Formular
  └─ Ruft scrape_willhaben("iPhone 14", max_pages=5) auf

3. Service (willhaben_scraper.py)
  ├─ GET willhaben.at/iad/...?keyword=iPhone+14&page=1
  ├─ Extraktion via __NEXT_DATA__ JSON (primär)
  ├─ Falls Bilder fehlen: HTML-Fallback + merge
  ├─ Falls JSON leer: HTML-Fallback
  ├─ Seiten 2-5 wiederholen (1s Pause)
  └─ Gibt Liste von Produkt-Dicts zurück

4. Controller (scraper.py)
  ├─ SearchQuery-Objekt anlegen → DB (flush)
  ├─ Für jedes Dict → Product-Objekt anlegen
  ├─ db.session.commit()
  └─ Redirect zu /scraper/products/42

5. Browser → GET /scraper/products/42

6. Controller lädt Product.query.filter_by(search_query_id=42)
  └─ übergibt an products.html → Browser zeigt Card-Grid

7. Nutzer klickt "Preisprognose"
  └─ GET /scraper/predict/42

8. Service (price_predictor.py)
  ├─ Preise extrahieren, Ausreißer filtern
  ├─ Polynomial Ridge Regression fitten
  └─ Preis bei Perzentile 50 vorhersagen

9. prediction.html zeigt Prognoseergebnis
```

12. Bekannte Bugs & Korrekturen

Im originalen Code wurden zwei Bugs identifiziert und behoben:

Bug 1: _parse_price() – Falsche Operationsreihenfolge

Datei: app/services/willhaben_scraper.py, Zeile ~40
Symptom: Preise wie "€ 1.234,56" wurden als 123456 (ohne Dezimalstelle) geparst
Ursache: replace(".", "") und replace(",", ".") wurden vor der Regex ausgeführt

Fehlerhafter Code:

```
# BUG: Die Ersetzungen passieren VOR dem Regex-Strip
cleaned = re.sub(r"[^\d,.]", "", text.replace(".", "").replace(",", "."))
# Bei "1.234,56": nach replace -> "1234.56", nach Regex -> "123456." -> 123456.0
```

Korrektur:

```
# FIX: Erst Regex, dann europäisches Format konvertieren
cleaned = re.sub(r"[^\d,.]", "", text).replace(".", "").replace(",", ".")
# Bei "1.234,56": nach Regex -> "1.234,56", nach replace -> "1234.56" -> 1234.56
```

Bug 2: _extract_next_data_products() – Falscher Beschreibungs-Fallback

Datei: app/services/willhaben_scraper.py, Zeile ~210
Symptom: Bei Inseraten ohne BODY_DYN-Attribut waren Titel und Beschreibung identisch
Ursache: Fallback für description war advert.get("description") – das gleiche Feld wie der Titel-Fallback

Fehlerhafter Code:

```
# BUG: Gleicher Fallback wie für heading_values
body_values = attr_map.get("BODY_DYN") or [advert.get("description") or ""]
```

Korrektur:

```
# FIX: Leerer String als Fallback - Titel und Beschreibung bleiben getrennt
body_values = attr_map.get("BODY_DYN") or [""]
```

13. Installation & Inbetriebnahme

13.1 Voraussetzungen

- Python 3.14 oder höher
- uv (empfohlen) oder pip

13.2 Installation mit uv

```
# Repository klonen
git clone https://github.com/kskgwyw-mij/scraper
cd scraper-main

# Abhängigkeiten installieren
uv sync

# Anwendung starten
uv run python run.py
```

13.3 Installation mit pip

```
# Virtuelle Umgebung erstellen
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# Abhängigkeiten installieren
pip install -r requirements.txt

# Anwendung starten
python run.py
```

13.4 Tests ausführen

```
pytest tests/ -v
```

13.5 Umgebungsvariablen

Variable	Standard & Beschreibung
FLASK_ENV	default – Konfiguration (development / testing / production)
SECRET_KEY	dev-secret-key-... – Unbedingt in Produktion ändern!
DATABASE_URL	sqlite:///scraper.db – Datenbankverbindung
LOG_LEVEL	INFO – Logging-Level (DEBUG / INFO / WARNING / ERROR)
PORT	5000 – HTTP-Port der Anwendung

Ende der Dokumentation

© 2024 Willhaben Scraper